SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5101

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

Total Marks: 50

Annexure A

Code of Conduct:

I J. LAKSHMI DEVI (192524300) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

1. Secure Password Storage in Web Applications
A web application stores user passwords and needs to protect them against database breaches.

A. Explain why storing plaintext passwords is insecure.
B. Suggest a secure method for storing passwords and justify your choice.

1. Secure Password Storage in Web Applications

Introduction

Password security is an important part of web application security. Most web applications require users to create accounts using usernames and passwords. These passwords protect personal information, financial details, and other sensitive data.

If passwords are not stored securely, hackers may steal them during database breaches. Therefore, web applications must use secure password storage techniques to protect user accounts and maintain data confidentiality.

A. Explain why storing plaintext passwords is insecure.

What is a Plaintext Password?

A plaintext password is a password stored in its original, readable format without any encryption or hashing.

For example, if a user creates the password Student@123, the application stores exactly the same password in its database.

If an attacker gains access to the database, the password can be read immediately.

Reasons Why Plaintext Passwords Are Insecure

1. Risk of Database Breaches

Web applications store user information in databases. If hackers exploit security vulnerabilities and access the database, they can easily obtain plaintext passwords.

This allows attackers to log in to user accounts without needing to crack or decrypt anything.

2. Unauthorized Account Access

When passwords are stored in plaintext, anyone who obtains database access can use them to enter user accounts.

Attackers may steal personal information, change account settings, or perform unauthorized transactions.

3. Password Reuse Attacks

Many users reuse the same passwords across multiple websites.

For example, a person may use the same password for email, social media, and online shopping.

If one website suffers a data breach, attackers may attempt to access the user's other accounts using the stolen password.

This attack is commonly called *credential stuffing*.

4. Insider Security Threats

Database administrators or employees with access to sensitive information may be able to view plaintext passwords.

A dishonest employee could misuse these passwords to access user accounts without permission.

5. Identity Theft and Financial Loss

Stolen passwords can be used to access banking applications, email accounts, and other personal services.

Attackers may steal identities, perform fraudulent transactions, or misuse confidential information.

6. Lack of Data Confidentiality

Confidentiality means protecting information from unauthorized access.

Plaintext password storage does not provide adequate confidentiality because passwords remain readable to anyone who accesses the database.

7. Damage to Organization Reputation

If a company experiences a password-related security breach, customers may lose trust in its services.

The organization may also face financial losses, legal consequences, and reputational damage.

Example

Consider a college website that stores student login details.

Username	Password
student101	Student@123
student102	College@456
student103	Welcome@789

If a hacker accesses this database, all three passwords are immediately visible.

Therefore, plaintext password storage creates a serious security risk.

B. Suggest a secure method for storing passwords and justify your choice.

Suggested Method: Salted Password Hashing Using Argon2id

One of the most secure methods for storing passwords is **salted password hashing using Argon2id**.

Argon2id is a modern password-hashing algorithm designed to protect passwords against offline guessing attacks.

Unlike encryption, hashing is a one-way process. The original password cannot be directly recovered from its hash.

What is Password Hashing?

Password hashing is the process of converting a password into a fixed-length value using a mathematical algorithm.

For example:

Original Password: Student@123

Stored Value: A password hash generated by Argon2id.

The application stores the hash instead of the original password.

During login, the system verifies the entered password against the stored hash.

What is Salting?

A salt is a unique, randomly generated value added to the password-hashing process.

Even if two users choose the same password, their stored password hashes will be different because each password uses a different salt.

Salting protects passwords against precomputed attacks, including rainbow-table attacks.

Steps Involved in Secure Password Storage

Step 1: User Registration

The user creates an account by entering a username and password.

Step 2: Salt Generation

The application generates a unique random salt using a cryptographically secure random number generator.

Step 3: Password Hashing

The password and salt are processed using the Argon2id hashing algorithm.

Step 4: Secure Database Storage

The application stores the password hash, salt, and necessary hashing parameters in the database.

The original password is not stored.

Step 5: User Login

When the user logs in, the application receives the entered password.

Step 6: Password Verification

The system verifies the entered password using the stored hash and hashing parameters.

Step 7: Access Decision

If verification succeeds, the user is authenticated. Otherwise, access is denied.

Advantages of Salted Password Hashing

1. Protects Original Passwords

The actual passwords are not stored in the database, reducing the risk of direct password exposure.

2. Provides Strong Security

Argon2id is designed to make large-scale password guessing attacks expensive.

3. Prevents Rainbow-Table Attacks

Unique salts make precomputed password-hash tables ineffective.

4. Reduces Database Breach Impact

Even if attackers steal the database, they must attempt to guess passwords rather than simply reading them.

5. Supports Secure Authentication

The application can verify user passwords without storing the original values.

6. Improves User Privacy

Secure password storage helps protect sensitive information and reduces the risk of unauthorized account access.

Additional Security Measures

Along with password hashing, web applications should implement:

1. **Strong password policies:** Encourage users to create long and difficult-to-guess passwords.
2. **Multi-factor authentication:** Require an additional verification factor to improve account security.
3. **HTTPS:** Protect passwords during transmission between the browser and server.
4. **Login rate limiting:** Restrict repeated login attempts to reduce automated guessing attacks.
5. **Secure database access:** Limit database permissions to authorized users and services.
6. **Regular security updates:** Fix known vulnerabilities in application software and server components.

Justification

Salted password hashing using Argon2id is recommended because it provides strong resistance against password-cracking attacks.

Argon2id is memory-hard, meaning attackers need significant memory and computational resources to test large numbers of password guesses.

Although weak passwords can still be cracked, Argon2id makes these attacks more difficult and expensive.

Therefore, it is more secure than plaintext storage or using fast general-purpose hashing algorithms such as MD5 and SHA-256 directly for password storage.

Conclusion

Secure password storage is essential for protecting web applications against database breaches and unauthorized access.

Storing passwords in plaintext exposes users to serious security threats, including identity theft, credential stuffing, and financial fraud.

Using salted password hashing with Argon2id provides a strong defense by protecting original passwords and making password guessing more difficult.

Therefore, organizations should implement secure password hashing along with multi-factor authentication, HTTPS, and appropriate access controls.

2. Digital Signature for Document Authentication

A company wants to ensure that official documents sent electronically are authentic and unaltered.

A. Explain how digital signatures can be used in this scenario.
B. Justify how this method ensures both integrity and non-repudiation.

2. Digital Signature for Document Authentication

Introduction

In modern organizations, electronic documents are widely used for business communication, financial transactions, legal agreements, and official correspondence.

However, documents transmitted through the internet may be modified, forged, or accessed by unauthorized individuals.

To overcome these security risks, organizations use digital signatures to verify the authenticity and integrity of electronic documents.

A digital signature is a cryptographic mechanism that helps confirm the identity of the signer and detect unauthorized modifications.

A. Explain how digital signatures can be used in this scenario.

What is a Digital Signature?

A digital signature is a mathematical technique used to verify the authenticity and integrity of digital information.

It is based on **asymmetric cryptography**, also known as public-key cryptography.

This method uses two mathematically related keys:

1. Private Key

The private key is kept secret by the document signer.

It is used to generate the digital signature.

2. Public Key

The public key is shared with receivers.

It is used to verify the digital signature.

Working Process of Digital Signatures

Step 1: Document Creation

The company prepares an official electronic document, such as a contract, invoice, agreement, or certificate.

For example, a company creates a PDF document containing an official business agreement.

Step 2: Hash Generation

The system applies a cryptographic hash function, such as SHA-256, to the document.

The hash function generates a fixed-length value representing the document's contents.

Even a small modification to the document will generally produce a completely different hash value.

Step 3: Digital Signature Generation

The company uses its private key and a digital signature algorithm to create a signature based on the document's hash.

Common digital signature algorithms include:

1. RSA-PSS
2. ECDSA
3. EdDSA

The resulting digital signature is associated with the electronic document.

Step 4: Document Transmission

The company sends the signed document to the receiver through email or another electronic communication system.

The digital signature may be embedded in the document or attached separately.

Step 5: Signature Verification

After receiving the document, the receiver uses the company's public key to verify the digital signature.

The receiver must also establish that the public key belongs to the claimed signer.

Digital certificates issued by trusted certificate authorities can help establish this relationship.

Step 6: Document Integrity Verification

The signature verification process checks whether the document matches the contents that were originally signed.

If the document has been modified after signing, verification will fail.

Step 7: Authentication Result

If verification succeeds and the public key is trusted, the receiver gains assurance that the document was signed using the corresponding private key and has not been altered.

Example

Consider a company sending an official employment offer letter to a candidate.

1. The HR department prepares the offer letter.
2. The company digitally signs the document using its private key.
3. The signed document is emailed to the candidate.
4. The candidate verifies the signature using the company's public key.
5. If the signature is valid, the candidate can confirm that the document has not been changed after signing.

If someone modifies the salary or joining date after the document is signed, the digital signature verification will fail.

This helps protect the document against unauthorized modifications.

Applications of Digital Signatures

1. Electronic Contracts

Digital signatures are used to authenticate agreements between businesses and customers.

2. Banking and Financial Transactions

Banks use digital signatures to authenticate certain electronic instructions and financial documents.

3. Government Documents

Government organizations use digital signatures for electronic certificates, official notices, and online services.

4. Educational Certificates

Universities can digitally sign certificates and academic documents to help prevent forgery.

5. Business Communication

Companies use digital signatures to authenticate invoices, reports, and official correspondence.

6. Software Distribution

Software developers use digital signatures to help users verify the authenticity and integrity of software packages.

B. Justify how this method ensures both integrity and non-repudiation.

Digital signatures provide important security properties, particularly integrity, authentication, and support for non-repudiation.

1. Integrity

Meaning:

Integrity ensures that information remains accurate, complete, and unchanged during storage or transmission.

Digital signatures help detect unauthorized modifications to electronic documents.

How Digital Signatures Ensure Integrity

Step 1: The sender generates a cryptographic hash of the original document.

Step 2: The sender creates a digital signature using the private key.

Step 3: The receiver obtains the document and its digital signature.

Step 4: The receiver verifies the signature using the sender's public key.

Step 5: If the document has been modified after signing, signature verification fails.

Example of Integrity

Suppose a company sends an invoice containing the following information:

Invoice Number: INV101

Product Amount: ₹15,000

Payment Due Date: 20 October 2026

The company digitally signs the invoice before sending it.

If an attacker changes the product amount from ₹15,000 to ₹50,000, the original digital signature will no longer verify successfully.

Therefore, the receiver can identify that the document has been modified.

Importance of Integrity

1. Protects documents against unauthorized changes.
2. Helps detect document tampering.
3. Maintains the accuracy of electronic records.
4. Improves trust in electronic communication.

5. Helps prevent fraudulent modifications to official documents.

2. Non-Repudiation

Meaning:

Non-repudiation provides evidence that a particular person or organization performed an action, making it difficult for them to deny that action later.

In digital signatures, non-repudiation relates to providing evidence that the signer signed a specific document.

How Digital Signatures Support Non-Repudiation

Step 1: Private Key Ownership

The signer possesses a private key that should remain confidential.

Step 2: Signature Creation

The signer uses the private key to generate a digital signature for the document.

Step 3: Public Key Verification

The receiver uses the corresponding public key to verify the signature.

Step 4: Identity Verification

A trusted digital certificate can associate the public key with the signer's identity.

Step 5: Evidence of Signing

A valid digital signature provides cryptographic evidence that the corresponding private key was used to sign the document.

When the signer's identity and private-key security are reliably established, this provides strong evidence of the signing action.

Example of Non-Repudiation

Consider a company signing a business agreement electronically.

1. Company A prepares a business contract.
2. An authorized representative of Company A digitally signs the contract.
3. Company B receives and verifies the signed contract.
4. Later, Company A disputes whether its representative signed the document.
5. The valid digital signature, trusted certificate, and supporting records can be used as evidence of the signing action.

This makes it more difficult for the signer to deny signing the document.

However, non-repudiation also depends on factors such as private-key protection, certificate validity, and reliable signing procedures.

Importance of Non-Repudiation

1. Provides evidence of document signing.

2. Reduces disputes between senders and receivers.
3. Supports accountability in electronic transactions.
4. Helps establish trust in digital agreements.
5. Provides useful evidence for audits and legal proceedings.

3. Authentication

Authentication verifies the identity of the signer.

Digital signatures help establish that a document was signed using the private key corresponding to a particular public key.

When that public key is reliably associated with the sender's identity, the receiver can authenticate the signer.

For example, a digitally signed government certificate can be verified using the issuing authority's trusted public key.

This helps recipients identify forged certificates.

Advantages of Digital Signatures

1. Improved Security

Digital signatures help protect electronic documents against forgery and unauthorized modification.

2. Document Authentication

They help receivers verify the identity of the document signer.

3. Data Integrity

They detect modifications made to documents after signing.

4. Non-Repudiation Support

They provide evidence that a particular private key was used to sign a document.

5. Time Saving

Documents can be signed and verified electronically without requiring physical meetings.

6. Cost Reduction

Digital signatures reduce printing, paper handling, and document transportation costs.

7. Faster Business Transactions

Organizations can exchange and authenticate electronic agreements quickly.

8. Environmental Benefits

Digital signatures support paperless operations and reduce paper consumption.

Limitations of Digital Signatures

Although digital signatures provide strong security, they have certain limitations.

1. Private Key Compromise

If an attacker steals the signer's private key, the attacker may be able to create fraudulent signatures.

2. Certificate Management

Organizations must manage digital certificates and ensure that public keys are trusted.

3. Technical Requirements

Digital signatures require suitable software and cryptographic infrastructure.

4. Key Expiration and Revocation

Certificates may expire or be revoked, requiring appropriate validation procedures.

5. No Automatic Confidentiality

Digital signatures do not encrypt the document contents. Encryption must be used separately when confidentiality is required.

Comparison Between Integrity and Non-Repudiation

Feature	**Integrity**	**Non-Repudiation**
Meaning	Ensures the document is unchanged	Provides evidence of the signing action
Main Purpose	Prevents undetected modification	Helps prevent denial of signing
Security Mechanism	Cryptographic hashing and signature verification	Private-key signing and identity verification
Verification	Checks whether the signed content has changed	Establishes evidence linking the signature to the signer
Example	Detecting changes to an invoice amount	Providing evidence that a company signed a contract

Conclusion

Digital signatures are an essential security mechanism for protecting electronic documents in modern organizations.

They use public-key cryptography and cryptographic hashing to authenticate signers and detect unauthorized changes.

Integrity ensures that documents remain unchanged after signing, while non-repudiation provides evidence of the signing action.

By combining digital signatures with trusted certificates and secure private-key management, organizations can improve the security, reliability, and accountability of electronic document exchange.

Therefore, digital signatures are widely used in banking, government services, educational institutions, and business transactions.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand